

Examen práctico - BiblioMobil

Informe de implementación, validaciones y evidencias de ejecución

Desarrollo de aplicaciones móviles multiplataforma

Datos de presentación	Información
Estudiante	Grease Duran Castro
Código	__202312217__
Fecha	15 / 09 / 2026
Duración del examen	4 horas (laboratorio), según el enunciado
Puntaje total de la rúbrica	20 puntos; no representa una nota obtenida
Proyecto de referencia	PharmaMobil (Sesiones 1 a 5)
Proyecto presentado	BiblioMobil · rama feature/clean-mvvm

Propósito del informe

Presentar la solución de biblioteca, explicar la organización del código y documentar las funciones observadas en Android. El contenido reúne la implementación, las ocho capturas solicitadas, las validaciones y los resultados de pruebas.

01 · Organización de la solución

Kotlin Multiplatform + Compose Multiplatform · Clean + MVVM

La primera versión permite registrar libros y lectores. El módulo de préstamos queda preparado en el dominio y muestra una pantalla en construcción. Los datos se almacenan en memoria mientras vive el proceso de la aplicación.

Ubicación	Responsabilidad
domain/model	Libro, Lector, Prestamo, DetallePrestamo y EstadoPrestamo; reglas y constantes del negocio.
domain/repository	Contratos LibroRepository y LectorRepository con registrar y listar suspend.
domain/usecase	Conversión y validación de entradas; cuatro casos de uso y helper resultadoDe.
data/repository	Listas en memoria, IDs desde 1, Mutex y latencia simulada.
presentation	Pantallas Compose, formularios, modelos de presentación y ViewModels con StateFlow.
di	Koin: repositorios por interfaz como single, casos de uso como factory y ViewModels.
navigation / theme	Una lista DESTINOS, navegación guardable y paletas clara y oscura propias.

Recorrido de un registro

La pantalla envía el texto al ViewModel; este invoca el caso de uso. El caso de uso valida y usa el contrato del repositorio. La implementación almacena el registro y devuelve el ID. El ViewModel actualiza StateFlow, limpia el formulario y recarga la lista.

Pantalla → ViewModel → Caso de uso → Contrato del repositorio

Koin conecta ese contrato con la implementación en memoria. El dominio no importa Compose, Android, iOS ni Koin.

androidApp inicia Koin desde MainApplication. iosApp lo inicia desde iOSApp.swift mediante initKoinlos(). Esta configuración iOS está escrita, pero su compilación no se ha comprobado en un Mac.

02 · Reglas y validaciones

Reglas tomadas de los anexos A y B del examen

Campo o modelo	Regla implementada
Libro: título y autor	Obligatorios; se guardan sin espacios al inicio ni al final.
Libro: año	Obligatorio, entero y entre 1450 y 2026.
Libro: ejemplares	Obligatorio, entero y no negativo. Menos de 3 activa requiereReposicion.
Lector: nombre y correo	Nombre obligatorio. Correo obligatorio y con formato válido.
Lector: teléfono	Opcional. Si se introduce, debe contener de 6 a 9 dígitos. En blanco se guarda como null.
DetallePrestamo	días entre 1 y 15. multaPorRetraso = días de retraso × 1.50.
EstadoPrestamo	Solicitado, Entregado, Devuelto y Vencido(diasRetraso).

Mensajes exactos por campo

Ejemplos: «El título es obligatorio», «El año debe ser un número entero», «El año debe estar entre 1450 y 2026» y «El teléfono debe tener entre 6 y 9 dígitos». Las pruebas verifican estos mensajes. El ViewModel recibe los errores y los asigna al formulario, sin repetir las reglas.

Comportamiento de la interfaz

El error del campo desaparece al escribir. Durante el registro se deshabilita el botón y se muestra «Registrando...». Al completar la operación se limpia el formulario, aparece la confirmación y se actualiza el catálogo. El botón se bloquea antes de iniciar la corrutina para impedir registros duplicados.

Formato y estados

Los libros muestran «1998 · 3 ejemplares» o «1998 · 1 ejemplar». El teléfono ausente se presenta como «No registrado». El catálogo distingue Cargando, Sin libros, Con libros y Error; este último ofrece Reintentar.

La conservación del formulario al rotar se comprobó en Android. El almacenamiento en memoria no implica persistencia: al terminar el proceso se pierden los registros.

03 · Inicio e identidad visual

Portada, accesos rápidos y modo de visualización.



Figura 1. Inicio en modo claro

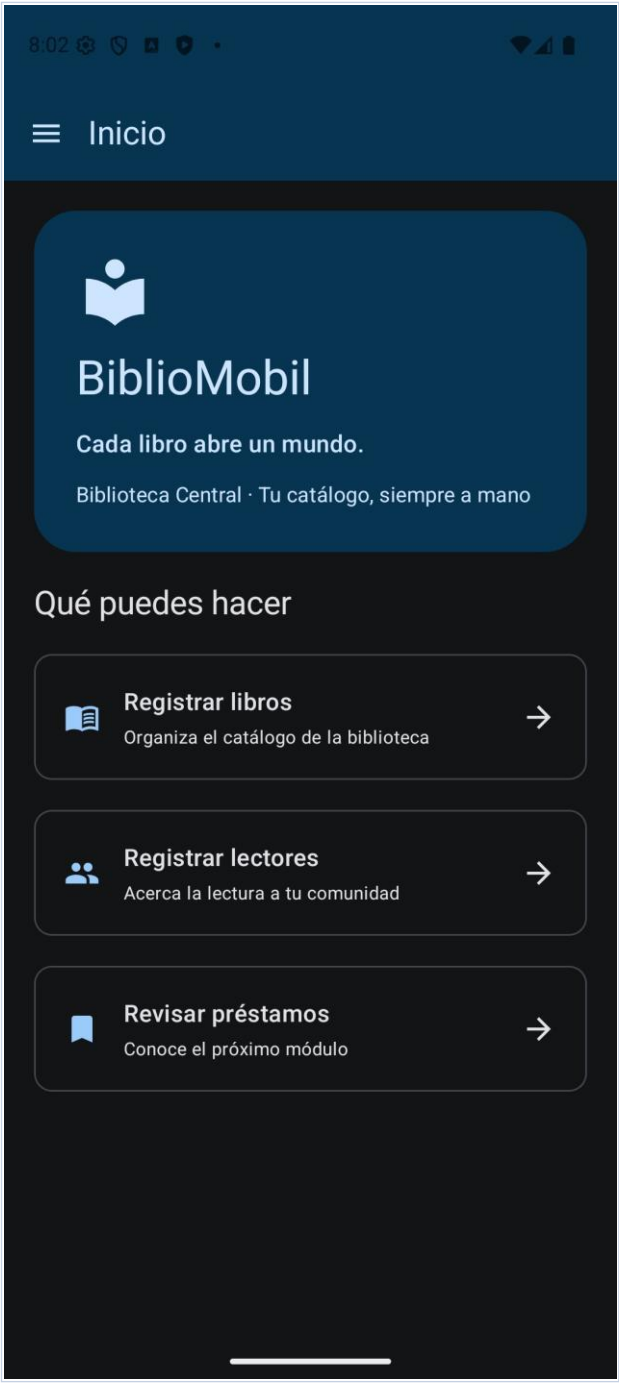


Figura 2. Inicio en modo oscuro

Marca BiblioMobil y accesos a Libros, Lectores y Préstamos, Paleta propia azul y ámbar. Se corrigió el contraste del título y del generados desde la lista de destinos. icono de menú en la barra superior.

04 · Catálogo de libros

Formulario con título, autor, año y ejemplares.



Figura 3. Catálogo vacío

El formulario permanece disponible y se presenta el estado «Sin Listado con confirmación y conteo. Rayuela, con 2 ejemplares, libros» con su mensaje de orientación. muestra el aviso «Pocos ejemplares».



Figura 4. Libros registrados

05 · Carga y recuperación de errores

Evidencias obtenidas con una simulación temporal permitida por el enunciado.



Figura 5. Estado Cargando

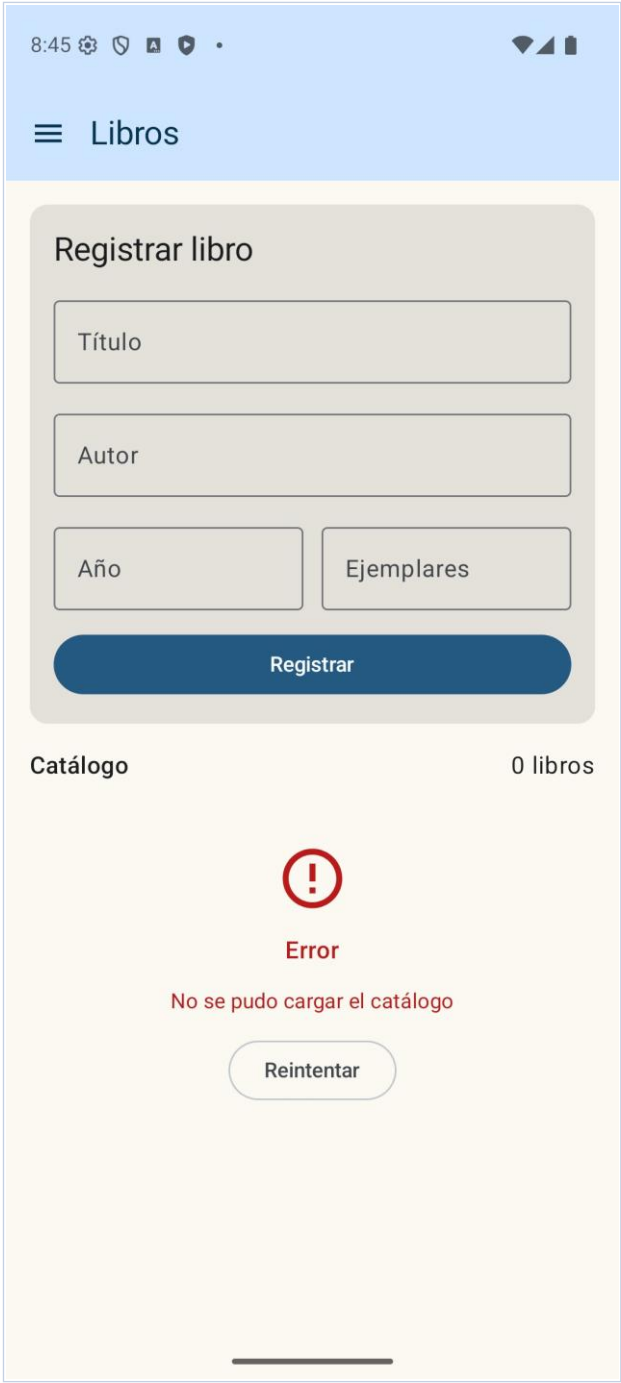


Figura 6. Estado Error

Indicador y texto mientras se consulta el catálogo. Para capturar Mensaje de fallo y botón Reintentar. Se provocó temporalmente este estado se amplió temporalmente la espera. una excepción; el código normal fue restaurado después.

06 · Lectores y préstamos

Registro de lectores y alcance de la primera versión.

7:33

Lectores

Registrar lector

Nombre

Correo

Teléfono

Opcional, entre 6 y 9 dígitos

Registrar

Lector "Elena" registrado correctamente

Cartera

2 lectores

greysi

greysi@gmail.com

987654346

Elena

elena@example.org

No registrado

Figura 7. Teléfono opcional

Elena fue registrada sin teléfono y la interfaz muestra «No registrado». Los datos visibles son registros de práctica.

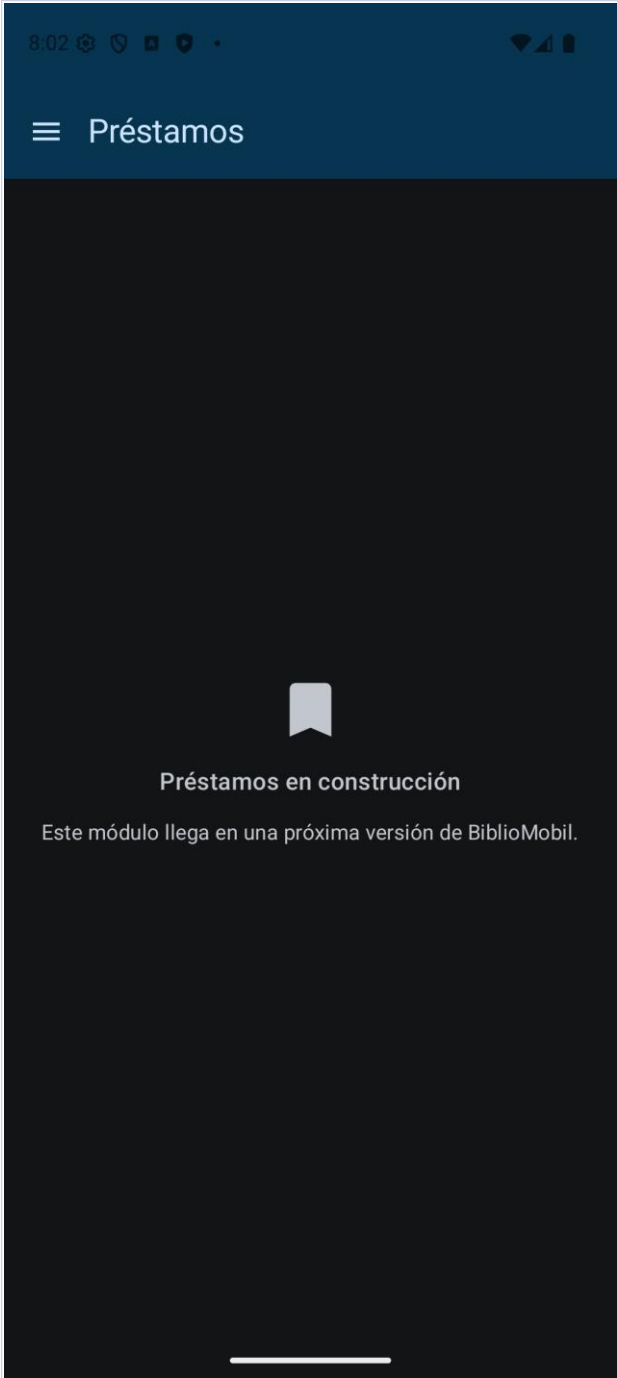


Figura 8. Préstamos en construcción

El módulo muestra icono Bookmark, título y descripción. No se exige registrar préstamos en esta versión.

Comprobaciones manuales

Se verificaron registro de libros y lectores, teléfono ausente, navegación y cambio de tema. Se escribió un formulario, se rotó el dispositivo y se comprobó su conservación. Dos pulsaciones consecutivas en Registrar añadieron un solo libro; el catálogo pasó de 1 a 2 registros.

Incidencias y resolución

La automatización de capturas tuvo problemas al leer el árbol de accesibilidad y reconocer el texto de carga. Las dos evidencias se obtuvieron después desde el emulador. Se restauró el archivo del repositorio desde el commit aprobado y la instalación final de la aplicación normal terminó correctamente.

Fuente de resultados: docs/PRUEBAS.log, docs/RESPUESTAS.md y los XML de shared/build/test-results. Las ejecuciones posteriores que muestran UP-TO-DATE reutilizan la verificación anterior; no representan 48 pruebas nueva.

07 · Fundamentos de la solución

Parte II · Respuestas incluidas en docs/RESPUESTAS.md

1. Llegada del backend REST

Añadiría `data/repository/LibroRepositoryRemoto.kt` y `LectorRepositoryRemoto.kt`, junto con el cliente HTTP y los DTO necesarios para convertir las respuestas del servidor en modelos del dominio. Cambiaría `di/AppModule.kt` para enlazar `LibroRepository` y `LectorRepository` con esas implementaciones y actualizaría las dependencias y la configuración de red. Quedarían intactos los modelos de `domain/model`, las interfaces de `domain/repository`, los cuatro casos de uso, los `ViewModels` y las pantallas: las capas exteriores dependen de los contratos del dominio y el dominio no conoce la fuente de datos. Las pruebas de esas capas pueden seguir usando repositorios falsos; añadiría pruebas para el acceso remoto.

2. Año y ejemplares como String

Recibir los campos como `String` permite distinguir un campo vacío, un texto que no representa un entero y un entero fuera del rango permitido, devolviendo el mensaje exacto de cada situación. Si `RegistrarLibroUseCase` recibiera `Int`, la pantalla o el `ViewModel` tendría que convertir antes de llamar al caso de uso y decidir qué hacer con valores como `""` o `"abc"`; eso repartiría la validación entre capas y podría duplicarla en otras interfaces. Por eso el caso de uso convierte y valida la entrada, mientras que el modelo conserva `anio` y ejemplares como enteros válidos.

3. Repositorio factory en lugar de single

Con `factory`, Koin crearía un repositorio nuevo cada vez que se resolviera esa dependencia. En esta aplicación, `RegistrarLibroUseCase` y `ListarLibrosUseCase` podrían recibir instancias distintas, cada una con su lista en memoria: el usuario registraría un libro y vería la confirmación, pero al recargar el catálogo ese libro no aparecería. Con `single`, ambos casos de uso comparten la misma instancia y consultan los mismos registros mientras vive el proceso de la aplicación; esto no equivale a guardar los datos permanentemente en disco.

Estas respuestas explican las decisiones del proyecto y sirven como apoyo para su exposición oral. Conviene comprender cada ejemplo y expresarlo con palabras propias.

08 · Correspondencia y entrega

Relación entre los ítems del examen y la implementación; no es una calificación.

Ítem	Evidencia en el proyecto
1. Proyecto y rama	Android/iOS, paquete pe.edu.upeu.bibliomobil, rama feature/clean-mvvm.
2. Dominio	Cinco modelos, constantes, require y dos contratos.
3. Casos de uso	Cuatro casos de uso, errores de validación y resultadoDe.
4. Datos	Dos repositorios en memoria con IDs, delay y Mutex.
5-6. Libros y Lectores	Formularios, StateFlow, estados, listas y validaciones.
7. Koin	Módulos comunes y de plataforma; inicialización Android e iOS.
8. Navegación e Inicio	DESTINOS compartido, menú lateral, accesos y Saver.
9. Identidad y componentes	Tema propio y componentes de campos, éxito y estado vacío.
10. Pruebas	48 casos aprobados y evidencias de ejecución.
Parte II y capturas	RESPUESTAS.md y ocho imágenes en docs/CAPTURAS.

Estado final documentado

La aplicación normal fue reinstalada y las ocho capturas se guardaron en el commit **643e5b6**. El historial incluye un commit por cada uno de los diez ítems. La simulación de error no forma parte del repositorio de datos final.

Verificación pendiente

La compilación del destino iOS debe comprobarse en un Mac con Xcode mediante **bash ./gradlew :shared:compileKotlinlosSimulatorArm64**. El aviso de pruebas iOS deshabilitadas en Windows no equivale a una compilación iOS aprobada.

Material de referencia: caso-biblioteca_hsnwtz6byt.pdf, anexos A-D y sección Entrega; proyecto BiblioMobil; docs/RESPUESTAS.md; docs/VERIFICACION.md; docs/CAPTURAS. Este PDF complementa esos archivos, no los sustituye.